



Course Code: CSE 2104
Course Title: Object Oriented Programming Lab

LABORATORY Open-Ended LabMANUAL

Department of Computer Science and Engineering
University of Liberal Arts Bangladesh
Spring 2026

Open-ended experiment

Course Title: Object Oriented Programming Lab	Section: 3
Course Code: CSE2104	Semester: Spring 2026
Total Marks: 40 (with Viva)	Submission Deadline: 23.04.2026
General Instructions: • This is an open-ended experiment. Students are expected to develop their own experiment; • Show each step of your experimental procedure, data, and calculations; • Discuss your results with relevant theories; • Originality of the work is a must; • Please refer to the assessment rubrics while preparing the report; • Symbols, notations and abbreviations carry their usual meanings.	

CO	Description	Domain/Model	Assessment Tools
CO1	Demonstrate the fundamental concepts of object-oriented programming	Cognitive / L3, Affective / L2, Psychomotor/L2	Open ended Labs
CO2	Apply the concepts of object-oriented programming for solving real-world problems	Cognitive / L3, Affective / L2, Psychomotor/L2	Open ended Labs

Problem:

Open-ended features:
• Use modern tools. • Use OOP features • Use GUI (Bonus) • Select any library management system that you prefer

Task No.	Corresponding COs	Marks

Case Study Lab Assignment: Smart Library Catalog System

Problem Scenario:

A university library wants to modernize its book cataloging system. They need a **digital catalog** that:

- Allows storing books with unique ISBNs.
- Supports multiple copies of the same book.
- Allows users to search for books by title, author, or genre.
- Tracks how many times a book has been borrowed (for analytics).

The library expects students to develop a system that leverages HashMap in **multiple intelligent ways**.

Learning Objectives:

- Practice using HashMap in various real-world scenarios.
- Understand nested or composite data structures (HashMap inside another HashMap, or with List/Set values).
- Reinforce object modeling and encapsulation (i.e., Book, Catalog, Library classes).
- Improve ability to design **efficient lookups and mappings**.

Key Requirements:

1. Model a Book Class:

- ISBN (String, unique)
- Title (String)
- Author (String)
- Genre (String)
- Total Copies (int)
- Borrow Count (int)

2. Implement a LibraryCatalog Class:

Use HashMap for these features:

- **Cataloging Books:**

java

CopyEdit

HashMap<String, Book> catalogByISBN;

- **Search Books by Title/Author/Genre:**

- Use additional HashMap<String, List<Book>> for quick search by:

- Title
- Author
- Genre
- **Borrow a Book (by ISBN):**
 - If available copies > 0:
 - Reduce available copies
 - Increase borrow count
 - Else:
 - Show “No copies available.”
- **Return a Book (by ISBN):**
 - Increase available copies

Challenge Extensions (for deeper thinking):

1. **Top N Borrowed Books:**
 - Display top 3 most borrowed books using borrow count.
 - Hint: Could involve sorting or PriorityQueue.
2. **Track Users (Optional Advanced Stretch):**
 - Use HashMap<String, List<String>> to track which user borrowed which books.
3. **Update Book Information:**
 - Allow updating a book’s genre or title and reflect changes in related maps.
4. **Serialization (Optional):**
 - Save and load catalog from file (to simulate persistence).

Sample Test Cases to Build:

- Add multiple books with different authors/genres.
- Borrow and return books.
- Search by title, author, genre.
- Try borrowing when no copies are available.
- Print top 3 borrowed books.
-

Student Deliverables:

1. Java Classes: Book.java, LibraryCatalog.java, LibraryApp.java (for main method).
2. Written responses (short answers) to:
 - How was HashMap used in your design?
 - What challenges did you face with nested HashMap or data integrity?
 - What are the time complexities of your search and borrow operations?
 -

○

Extend the **Library Catalog System** to:

- Persist and retrieve data from **files** or a **database**.
- Implement abstract methods defined in **interfaces and abstract classes**.
- Follow principles of **OOP**: encapsulation, abstraction, separation of concerns.

Design Overview

Let's define:

1. CatalogPersistence (Interface)

- For abstraction over storage (File / DB).
- Students must implement this.

2. AbstractLibraryCatalog (Abstract Class)

- Defines high-level structure.
- Uses the interface `CatalogPersistence` for saving/loading data.
- Contains common fields like the `bookMap`.

Interface: `CatalogPersistence.java`

```
package library.persistence;

import library.models.Book;
import java.util.List;
import java.util.Map;

public interface CatalogPersistence {
    void saveCatalog(Map<String, List<Book>> catalog) throws Exception;
    Map<String, List<Book>> loadCatalog() throws Exception;
}
```

- `Map<String, List<Book>> catalog`: the key is ISBN, values are multiple copies.
- Students can implement this via:
 - **FileCatalogPersistence**: using Java File I/O (e.g., serialization, JSON, CSV).
 - **DatabaseCatalogPersistence**: using JDBC and SQL.

Abstract Class: AbstractLibraryCatalog.java

```
package library.core;

import library.models.Book;
import library.persistence.CatalogPersistence;

import java.util.*;

public abstract class AbstractLibraryCatalog {

    protected Map<String, List<Book>> catalog = new HashMap<>();
    protected CatalogPersistence persistence;

    public AbstractLibraryCatalog(CatalogPersistence persistence) {
        this.persistence = persistence;
    }

    // Common method to add a book
    public void addBook(Book book) {
        catalog.computeIfAbsent(book.getIsbn(), k -> new
ArrayList<>()).add(book);
    }

    // Abstract methods to be implemented
    public abstract void searchByTitle(String title);
    public abstract void searchByAuthor(String author);
    public abstract void searchByGenre(String genre);

    public abstract void borrowBook(String isbn);

    // Common persistence operations
    public void saveCatalog() {
        try {
            persistence.saveCatalog(catalog);
        } catch (Exception e) {
            System.out.println("Error saving catalog: " + e.getMessage());
        }
    }

    public void loadCatalog() {
        try {
            catalog = persistence.loadCatalog();
        } catch (Exception e) {
            System.out.println("Error loading catalog: " + e.getMessage());
        }
    }
}
```

Supporting `Book` Class (Skeleton)

Just in case you haven't given this to students, here's a clean version:

```

package library.models;

public class Book {
    private String isbn;
    private String title;
    private String author;
    private String genre;
    private int borrowCount;

    public Book(String isbn, String title, String author, String genre) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
        this.genre = genre;
        this.borrowCount = 0;
    }

    public void borrow() {
        borrowCount++;
    }

    // Getters and setters

    public String getIsbn() {
        return isbn;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }

    public String getGenre() {
        return genre;
    }

    public int getBorrowCount() {
        return borrowCount;
    }
}

```

What Students Will Implement

1. File or DB Persistence:

They'll implement `CatalogPersistence`:

```
// FileCatalogPersistence.java
```

```

public class FileCatalogPersistence implements CatalogPersistence {
    // Implement saveCatalog and loadCatalog using FileOutputStream /
    ObjectOutputStream
}
// DatabaseCatalogPersistence.java
public class DatabaseCatalogPersistence implements CatalogPersistence {
    // Use JDBC to save/load data to/from SQL database
}

```

2. Concrete Class extending AbstractLibraryCatalog

```

public class LibraryCatalogImpl extends AbstractLibraryCatalog {

    public LibraryCatalogImpl(CatalogPersistence persistence) {
        super(persistence);
    }

    @Override
    public void searchByTitle(String title) {
        // implement logic to search catalog
    }

    @Override
    public void searchByAuthor(String author) {
        // implement logic
    }

    @Override
    public void searchByGenre(String genre) {
        // implement logic
    }

    @Override
    public void borrowBook(String isbn) {
        List<Book> books = catalog.get(isbn);
        if (books != null && !books.isEmpty()) {
            books.get(0).borrow(); // assume borrowing first copy
            System.out.println("Book borrowed.");
        } else {
            System.out.println("Book not available.");
        }
    }
}

```

Suggested Learning Flow for Students

1. Implement Book and LibraryCatalogImpl.
2. Implement FileCatalogPersistence using serialization or text files.
3. Optionally, implement DatabaseCatalogPersistence using JDBC + SQL (for advanced students).
4. Write a Main class to demonstrate:

- Adding/searching/borrowing books
- ⊖ Saving and loading from storage
-

Suggested Package Structure

```
library/  
├── core/  
│   └── AbstractLibraryCatalog.java  
├── models/  
│   └── Book.java  
├── persistence/  
│   ├── CatalogPersistence.java  
│   ├── FileCatalogPersistence.java  
│   └── DatabaseCatalogPersistence.java  
├── impl/  
│   └── LibraryCatalogImpl.java  
└── Main.java
```